

On the Erdős Discrepancy Problem: Multiplicative Sequences and Finite Bounds

Charles EDOU NZE*

August 11, 2026

Abstract

We present a rigorous algebraic and combinatorial analysis of the Erdős Discrepancy Problem. The document outlines strict axiomatic foundations, details contextual literature on Boolean satisfiability approaches, establishes key lemmas on multiplicative sequences, and outlines an autoformalization architecture for the Lean 4 theorem prover.

Contents

*Charles EDOU NZE, chercheur indépendant

1 Axiomatic Definitions and Context

Definition 1.1. Let $(x_n)_{n \in \mathbb{N}}$ be an infinite sequence with elements in $\{-1, +1\}$. For any positive integer $d \in \mathbb{Z}_{>0}$ and any positive integer $k \in \mathbb{Z}_{>0}$, the discrepancy $D(d, k)$ associated with homogeneous arithmetic progressions of step d and length k is defined by:

$$D(d, k) = \left| \sum_{i=1}^k x_{i \cdot d} \right|$$

Definition 1.2. A sequence $(x_n)_{n \in \mathbb{N}}$ with elements in $\{-1, +1\}$ is said to be completely multiplicative if for all integers $a, b \in \mathbb{Z}_{>0}$, the property $x_{a \cdot b} = x_a \cdot x_b$ holds unconditionally.

The Erdős Discrepancy Conjecture proposes that for any constant $C \in \mathbb{Z}_{>0}$ and any sequence $(x_n)_{n \in \mathbb{N}}$ with values in $\{-1, +1\}$, there exist positive integers d and k such that $D(d, k) > C$. The assertion implies that it is impossible to bound the discrepancy of all homogeneous arithmetic progressions in any arbitrary binary sequence.

1.1 Contextual Literature Research

A substantial body of recent work focuses on bounding the maximum sequence length before the discrepancy is forced to strictly exceed a threshold C . For instance, researchers formulated the case $C = 2$ as a Boolean satisfiability (SAT) instance, generating sequences of length 1160 and concluding that no sequence of length 1161 exists with discrepancy bounded by 2. For completely multiplicative sequences (CMSs), specific studies prove that any CMS of size 127,646 or more necessarily exhibits a discrepancy of at least 4. Additionally, Terence Tao conclusively proved that the discrepancy is infinite for any sequence mapping to $\{-1, +1\}$, utilizing Fourier-analytic reductions to entirely multiplicative stochastic functions and leveraging a logarithmically averaged variant of the Elliott conjecture.

2 Proof Strategy and Lemma Isolation

To analyze the structural boundaries forcing the discrepancy to exceed a threshold C , the global problem is partitioned into two distinct lemmas. We consider the specific case of completely multiplicative sequences restricted to prime indices.

Lemma 2.1. Let $(x_n)_{n \in \mathbb{N}}$ be a completely multiplicative sequence taking values in $\{-1, +1\}$. If $x_2 = 1$ and $x_3 = 1$, then $x_6 = 1$.

Proof. By Definition 1.2, a completely multiplicative sequence $(x_n)_{n \in \mathbb{N}}$ must satisfy the algebraic relation $x_{a \cdot b} = x_a \cdot x_b$ for all positive integers a, b . We consider the index $n = 6$. The canonical prime factorization of 6 is exactly 2×3 . We apply the completely multiplicative property by setting $a = 2$ and $b = 3$. This yields the strict equality:

$$x_6 = x_{2 \cdot 3} = x_2 \cdot x_3$$

By the initial hypotheses of the lemma, we have explicitly set the values $x_2 = 1$ and $x_3 = 1$. Substituting these discrete values into the equation produces:

$$x_6 = 1 \cdot 1$$

Performing the arithmetic multiplication explicitly:

$$1 \cdot 1 = 1$$

Therefore, we deduce unequivocally that $x_6 = 1$. This structural constraint demonstrates how fixing values at prime indices rigidly determines the sequence's values at composite indices. $\square$

Lemma 2.2. *Consider the sequence segment bounded to indices $n \in \{1, 2, 3, 4, 5, 6\}$. If $(x_n)_{n \in \mathbb{N}}$ is completely multiplicative and $x_1 = 1, x_2 = 1, x_3 = 1, x_4 = 1, x_5 = 1, x_6 = 1$, the maximal discrepancy for $d = 1$ and $k = 6$ is 6.*

Proof. Let $d = 1$ and $k = 6$. We calculate the summation explicitly term by term based on Definition 1.1:

$$\sum_{i=1}^6 x_{i,1} = x_1 + x_2 + x_3 + x_4 + x_5 + x_6$$

Substituting the given fixed values of the sequence into the sum:

$$\sum_{i=1}^6 x_{i,1} = 1 + 1 + 1 + 1 + 1 + 1$$

We perform the iterative addition step-by-step:

$$1 + 1 = 2$$

$$2 + 1 = 3$$

$$3 + 1 = 4$$

$$4 + 1 = 5$$

$$5 + 1 = 6$$

Thus, the sum rigorously evaluates to 6. Finally, we apply the absolute value function:

$$|6| = 6$$

This proves that the discrepancy reaches exactly 6, which strictly exceeds the threshold $C = 2$ and bounds the maximum homogeneous subsequence under these rigid constraints. $\square$

3 Autoformalization Architecture

The subsequent block presents the explicit Lean 4 type structures needed to encode the definitions and lemmas into a formal verification engine.

```
import Mathlib.Data.Nat.Basic
import Mathlib.Tactic.Omega
import Mathlib.Tactic.Ring

-- Axiomatic definition of an infinite sequence of signs
def SignSeq := Nat -> Int

-- Property bounding the values to -1 and 1
```

```

def IsBinarySeq (x : SignSeq) : Prop :=
  forall n, n > 0 -> x n = 1 /\ x n = -1

-- Property defining completely multiplicative sequences
def IsCompletelyMultiplicative (x : SignSeq) : Prop :=
  forall a b, a > 0 -> b > 0 -> x (a * b) = x a * x b

-- Axiomatic definition of the discrepancy sum
def DiscrepancySum (x : SignSeq) (d k : Nat) : Int :=
  -- Assuming an abstract sum function sum_seq exists for indices 1
  -- to k
  sorry

-- Complete demonstration of Lemma 2.1
lemma completely_multiplicative_6 (x : SignSeq) (hm :
  IsCompletelyMultiplicative x)
  (h2 : x 2 = 1) (h3 : x 3 = 1) : x 6 = 1 := by
  have h_mult := hm 2 3 (by omega) (by omega)
  have h_six : 2 * 3 = 6 := by omega
  rw [h_six] at h_mult
  rw [h2, h3] at h_mult
  have h_one : 1 * 1 = (1 : Int) := by ring
  rw [h_one] at h_mult
  exact h_mult

-- General Theorem (Erdos Discrepancy Property)
theorem erdos_discrepancy (x : SignSeq) (h_bin : IsBinarySeq x) (C
  : Nat) :
  Exists (fun d => Exists (fun k => d > 0 /\ k > 0 /\
    DiscrepancySum x d k > C /\ DiscrepancySum x d k < -C)) :=
  by
  sorry

```